

Break & Continue

Break and continue are jump statements that alter the normal flow of loops (and break also works with switch).

When to use:

Break - use it when you need to exit a loop entirely before its natural end condition is reached (ex. found what you were searching for, no need to keep looping).

Continue - use it when you want to skip the current iteration and move to the next one, without exiting the whole loop.

How to use:

A) break - exits the loop immediately.

```
for (int i=1; i<=10; i++) {  
    if (i == 5) {  
        break;  
    }  
    s.o.p(i);  
}
```

//prints: 1 2 3 4

B) Continue - skips to the next iteration.

Ex:

```
for(int i=1; i<=10; i++){  
    if(i%2 == 0){  
        continue;  
    }  
    S.O.P(i);  
}
```

// print: 1 3 5 7 9

Where to use:

- * Inside any loop (for, while, do-while) - anywhere you need early exit to selective skipping.
- * break is also used inside switch ~~for~~ ~~see~~ to stop fall-through.
- * Common with searching: once a match is found, break avoids wasting time checking the rest.

Why:

- * Continue keeps code flatter/more readable vs. nesting remaining logic inside else.
- * Break/Continue apply only to the innermost enclosing loop - simple, predictable rule.
- * To break an outer loop from a nested loop, use a labeled break.

Ex:

```
outer:  
for(...){  
    for(...){  
        if(condition) break outer;  
    }  
}
```

- * Continue in a for loop still runs the update step (i++) and re-checks the condition
- only skips the code between continue and the iteration's end.

Real-life Example:

Scenario

statement

Stop searching once contact found	break
Skip absent students during roll call	Continue
Found item in nested row/column search	Labeled break

Answer (D & D)